

Technical Reference: Text Processing

Optional details on Unicode, normalization, tokenization, and reproducible preprocessing

Dr. A. Belcaid

Table of contents

1 From language to computation **2**
1.1 What makes language difficult? . . . 3
1.2 A task changes what counts as signal . . . 4

2 The NLP pipeline **4**

3 Text has layers **4**
3.1 Bytes and encodings . . . 5
3.2 Code points are not necessarily visible characters . . . 5

4 Unicode normalization **6**
4.1 Canonical decomposition and composition . . . 6
4.2 Compatibility normalization is more aggressive . . . 7
4.3 Normalization is not “remove everything unusual” . . . 7

5 Tokens and tokenization **8**
5.1 Tokenization is a contract . . . 8
5.2 The same sentence supports different valid tokenizations . . . 9
5.3 Offsets make tokens useful . . . 9
5.4 A preview of subword tokenization . . . 10

6 Regular expressions as bounded recognizers **10**
6.1 Why `\w+` is not a universal tokenizer . . . 11

7 From rules to an industrial tokenizer **12**
7.1 Compare tokenizers with adversarial examples . . . 12

8 A reproducible preprocessing pipeline **13**
8.1 Invariants worth testing . . . 13
8.2 Keep policy separate from mechanism . . . 14

9 Worked case study: a noisy multilingual review **14**
9.1 Step 1: preserve and inspect 14
9.2 Step 2: choose a normalization view 15
9.3 Step 3: tokenize with protected spans 15
9.4 Step 4: retain evidence 15

10 Summary **15**

11 Exercises **16**
11.1 Conceptual questions 16
11.2 Unicode investigation 16
11.3 Tokenizer experiment 16
11.4 Pipeline design 17

References **17**

Learning objectives

After studying this chapter, you should be able to:

1. explain why an NLP system must make explicit decisions about linguistic units;
2. distinguish bytes, Unicode code points, grapheme clusters, and tokens;
3. choose and justify an appropriate Unicode normalization policy;
4. design tokenization rules for a stated task and domain;
5. use regular expressions for bounded, testable text-processing problems; and
6. build a preprocessing pipeline that preserves provenance and character offsets.

1 From language to computation

Natural language processing (NLP) studies computational methods for working with human language. Its systems may **analyze** language, as in sentiment classification; **retrieve** information, as in search; **transform** language, as in translation; or **generate** language, as in a dialogue system. What these tasks share is not a single algorithm. They share a difficult interface between human communication and discrete computation.

A computer receives bytes. A reader perceives letters, words, intentions, references, and social context. An NLP pipeline builds intermediate representations that connect those two descriptions.

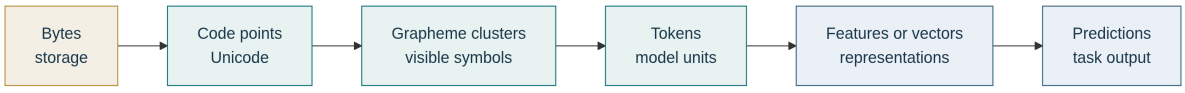


Figure 1: The representation ladder: each stage preserves some distinctions and removes others.

The arrows in Figure 1 are **design decisions**, not neutral conversions. Lowercasing removes distinctions. Tokenization inserts boundaries. A vocabulary maps strings to integer identifiers. Once a distinction has been discarded, a later model cannot recover it from the processed input alone.

> **! Central principle**
>
> Text preprocessing is not generic cleanup. It is the first layer of modeling: it defines what information the rest of the system can see.

1.1 What makes language difficult?

Language is productive, context-dependent, and ambiguous. The same surface form can support several analyses, while different surface forms can express nearly the same intent. Consider:

> I saw the student with the telescope.

The prepositional phrase *with the telescope* can describe the instrument used for seeing, or the student who possessed the telescope. Nothing about splitting the sentence at spaces resolves that structural ambiguity.

Ambiguity appears at several interacting levels:

Table 1: Linguistic levels are analytical lenses, not independent processing boxes.

Level	Central question	Example
Morphology	How are meaningful pieces combined inside a word?	*un-help-ful-ness*
Syntax	Which words form constituents and dependencies?	Who has the telescope?
Semantics	What entities, events, and relations are expressed?	*bank* as an institution or river edge
Pragmatics	What does the speaker intend in context?	“Can you open the window?” as a request
Discourse	How do sentences connect across a text?	What does *it* refer to?

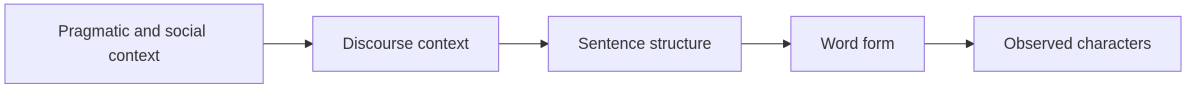


Figure 2: Interpretation is constrained by nested sources of context.

Table 1 and Figure 2 should not be read as a fixed recipe in which morphology always runs before syntax. Modern models often learn several levels jointly. The useful lesson is that a visible string underdetermines its interpretation. Text processing must therefore preserve distinctions relevant to the eventual task.

1.2 A task changes what counts as signal

Suppose the input is:

> AMAZING service!!! *(followed by two heart-eyes emoji)*

For topic classification, case and repeated punctuation may be dispensable. For sentiment or author-style analysis, they are evidence. For a moderation system, the emoji may be crucial. There is no universally "clean" version of this sentence; there are only transformations justified—or not justified—by a task.

2 The NLP pipeline

An NLP system is more than a model. It is a chain connecting a population and a real communication setting to a measurable output.

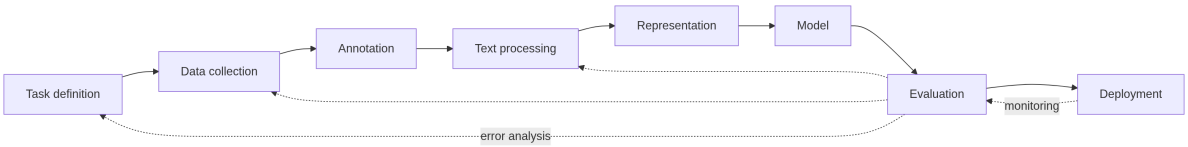


Figure 3: A practical NLP pipeline. Feedback arrows matter because error analysis can change every earlier decision.

Each stage introduces assumptions:

- **Task definition:** What decision will the output support, and for whom?
- **Data collection:** Which languages, dialects, genres, and time periods are represented?
- **Annotation:** Is the label observable, or does it require judgment? Do annotators agree?
- **Text processing:** Which distinctions are preserved, merged, or deleted?
- **Representation and modeling:** What patterns can the chosen model express?
- **Evaluation:** Does the test set reflect the deployment population and failure costs?
- **Deployment:** Can inputs drift? Can users adapt to or exploit the system?

> ⚠ **Pipeline errors compound**
>
> If a tokenizer removes a negation marker, no classifier can use it. If character offsets drift during normalization, a correct named-entity prediction may be highlighted over the wrong span. A high model score does not repair a broken data interface.

3 Text has layers

To process text reliably, we must distinguish four units that everyday speech often calls a "character."

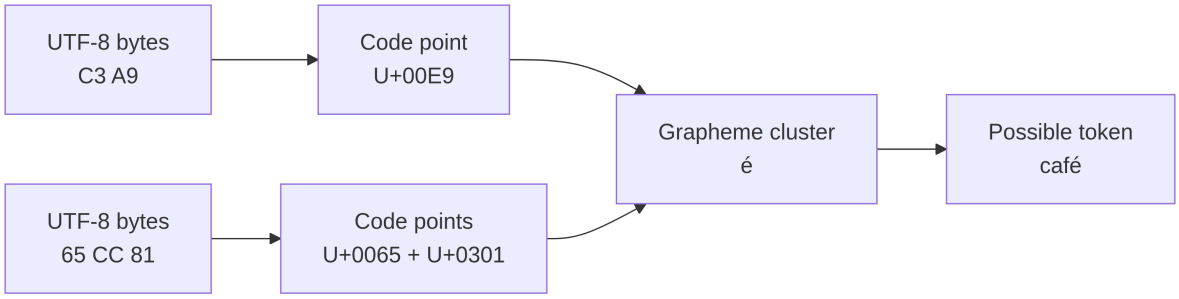


Figure 4: Four meanings commonly hidden behind the word character. The example é may be stored in more than one canonically equivalent way.

3.1 Bytes and encodings

A **byte** is an integer from 0 to 255. Files and network messages are byte sequences. An **encoding** specifies how those bytes represent text. UTF-8 is the dominant Unicode encoding on the web and uses one to four bytes per code point. ASCII characters keep their familiar one-byte values, while other characters use multibyte sequences (Jurafsky and Martin 2026).

Decoding must precede linguistic processing:

```
payload = b"caf\xc3\xa9"
text = payload.decode("utf-8")
assert text == "café"
```

Using the wrong encoding can produce an error or *mojibake*: valid-looking but corrupted text such as `cafÃ©`. Once corruption is silently accepted and the original bytes are discarded, recovery may be ambiguous.

3.2 Code points are not necessarily visible characters

Unicode assigns an abstract integer called a **code point** to each encoded character. Python's `ord()` exposes that integer:

```
for char in "NLP":
    print(char, f"U+{ord(char):04X}")
```

```
N U+004E
L U+004C
P U+0050
```

But a visible symbol can contain several code points. The decomposed spelling e followed by COMBINING ACUTE ACCENT is perceived as one é. Emoji sequences make this distinction even clearer: a family emoji can contain several people joined by ZERO WIDTH JOINER characters.

Unicode calls a user-perceived unit an **extended grapheme cluster** and specifies default grapheme, word, and sentence boundary rules in Unicode Standard Annex #29 (Unicode Consortium 2025b). These rules are essential for cursor motion, truncation, and display, but they do not by themselves define the linguistic tokens required by every NLP task.

> **A practical debugging habit**
>
> When two strings look equal but compare unequal, inspect `repr(text)`, the code points, and the Unicode names before blaming the model.

```
import unicodedata

def inspect_text(text: str) -> None:
    for char in text:
        print(
            f"U+{ord(char):04X}",
            unicodedata.name(char, "<unnamed>"),
            repr(char),
        )
```

4 Unicode normalization

Two strings can be canonically equivalent yet have different code-point sequences. Unicode normalization maps equivalent sequences into consistent forms so that comparison, indexing, and counting behave predictably (Unicode Consortium 2025a).

4.1 Canonical decomposition and composition

For many accented characters, Unicode permits both:

- a **precomposed** code point: é = U+00E9;
- a **decomposed** sequence: e + ◌́ = U+0065 U+0301.

The normalization forms differ along two axes:

Table 2: The four Unicode normalization forms. The `K` marks compatibility normalization.

Form	Equivalence used	Result
NFD	Canonical	Decomposed
NFC	Canonical	Decomposed, then recomposed where possible
NFKD	Compatibility	Decomposed
NFKC	Compatibility	Decomposed, then recomposed where possible

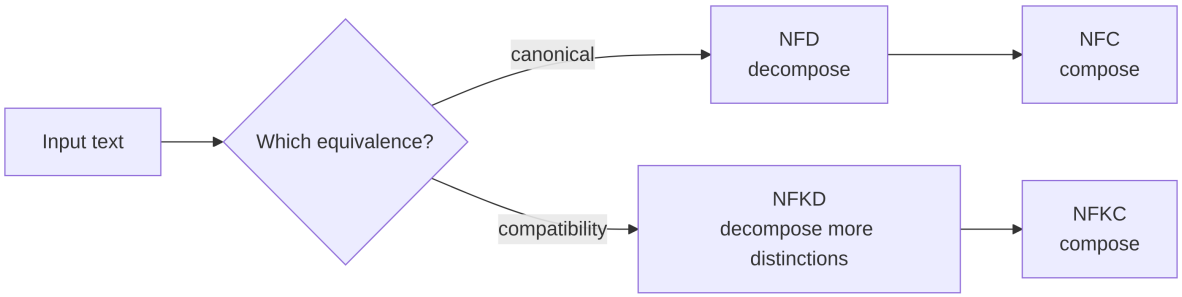


Figure 5: Canonical and compatibility normalization answer different questions.

NFC is a sensible default for much stored natural-language text: it respects canonical equivalence while usually retaining compact composed forms. Python implements the standard operation directly:

```
import unicodedata

left = "café"              # U+00E9
right = "cafe\N{COMBINING ACUTE ACCENT}"

assert left ≠ right
assert unicodedata.normalize("NFC", left) ==
 ↪  unicodedata.normalize("NFC", right)
```

4.2 Compatibility normalization is more aggressive

NFKC and NFKD also collapse compatibility distinctions. For example, NFKC can map the ligature fi to `fi`, the circled digit ① to `1`, and full-width Latin forms to their ordinary counterparts. This is useful for search keys and identifiers, but it is **not lossless**. The Unicode specification explicitly warns against blindly applying compatibility normalization to arbitrary text because formatting distinctions may carry meaning (Unicode Consortium 2025a).

> **! Normalize for a purpose**
>
> Choose a policy by asking which equivalences the application needs:
>
> - archival text: preserve the original and derive a normalized view;
> - canonical comparison: often NFC;
> - search or identifier matching: perhaps NFKC plus a documented case policy;
> - stylistic, mathematical, or forensic analysis: retain distinctions that compatibility normalization would erase.

4.3 Normalization is not “remove everything unusual”

Lowercasing, accent removal, punctuation deletion, whitespace collapsing, and Unicode normalization are separate operations. Each has different semantics. Consider Turkish *I*, *İ*, *ı*, and *i*; case operations are language-sensitive. Consider Arabic diacritics: removing

them may reduce sparsity but can also erase lexical or grammatical information. A robust system names each transformation rather than hiding them inside a function called `clean_text`.

The order also matters. A useful pipeline records it explicitly:

```python
from dataclasses import dataclass
import re
import unicodedata

@dataclass(frozen=True)
class NormalizationPolicy:
    unicode_form: str = "NFC"
    collapse_whitespace: bool = True
    casefold: bool = False

def normalize_text(text: str, policy: NormalizationPolicy) -> str:
    result = unicodedata.normalize(policy.unicode_form, text)
    if policy.casefold:
        result = result.casefold()
    if policy.collapse_whitespace:
        result = re.sub(r"\s+", " ", result).strip()
    return result
```

This code is intentionally conservative. It does not delete punctuation, strip accents, or guess at malformed encodings.

5 Tokens and tokenization

A **token** is a unit selected for a computational purpose. A **token type** is a distinct vocabulary entry, while a **token instance** is an occurrence in a text. In `time flies like an arrow`, there are five token instances; if all are distinct, there are five types.

Tokenization is often introduced as "splitting text into words." That definition fails quickly:

```
don't    state-of-the-art    3.14    06/08/2026
anass@example.org    https://example.org/a?x=1    #NLP
والكتاب    中文分词    woman-technologist emoji
```

Spaces do not tell us whether `don't` is one unit or two, whether hyphens are internal, or where words begin in scripts that do not use spaces as English does. Jurafsky and Martin treat tokenization as the problem of identifying the units that a later model will operate on, not merely detecting whitespace (Jurafsky and Martin 2026).

5.1 Tokenization is a contract

Before implementing a tokenizer, state the contract:

1. **Task:** What will consume the tokens?

2. **Domain:** News, source code, clinical notes, social media, or legal text?
3. **Languages and scripts:** Which writing systems and orthographic conventions occur?
4. **Protected spans:** Should URLs, email addresses, numbers, or emoji remain intact?
5. **Reversibility:** Must tokens reconstruct the original string?
6. **Alignment:** Must predictions map back to exact source offsets?

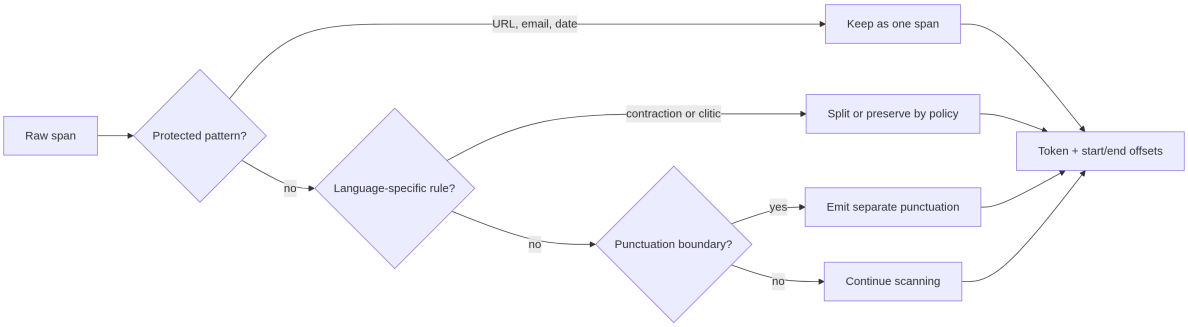


Figure 6: Tokenization turns a task specification into explicit boundary decisions.

5.2 The same sentence supports different valid tokenizations

Consider `I can't visit New York.`:

Purpose	Possible tokens	Reason
Surface display	`I`, `can't`, `visit`, `New`, `York`, `.`	Preserve familiar written words
Syntactic analysis	`I`, `ca`, `n't`, `visit`, `New`, `York`, `.`	Expose the negation clitic
Named-entity model	`I`, `can`, `'`, `t`, `visit`, `New`, `York`, `.`	Model-specific vocabulary rules
Gazetteer lookup	`I`, `can't`, `visit`, `New York`, `.`	Preserve a multiword entity candidate

None is universally correct. Correctness is measured against the declared contract and downstream behavior.

5.3 Offsets make tokens useful

A production token should usually store its source interval:

```python
from dataclasses import dataclass

@dataclass(frozen=True)
class Token:
    text: str
    start: int
    end: int
```

```
source = "NLP is fun!"
token = Token(text="NLP", start=0, end=3)
assert source[token.start:token.end] == token.text
```

Offsets support highlighting, annotation, debugging, and error analysis. If normalization changes string length, offsets in the normalized text no longer automatically identify the same positions in the source. Preserve the original or maintain an alignment map.

5.4 A preview of subword tokenization

Word vocabularies face an open-vocabulary problem: new names, inflections, compounds, and spelling variants appear continually. Character models avoid unknown words but create long sequences. **Subword tokenization** occupies the space between them, representing frequent strings as single units and rarer strings as compositions.

Sennrich, Haddow, and Birch adapted byte-pair encoding (BPE) for neural machine translation, showing that rare words can be modeled as subword sequences rather than mapped to a single unknown token (Sennrich et al. 2016). The broader history includes characters, morphemes, learned subwords, and bytes; no granularity is optimal for all applications (Mielke et al. 2021).

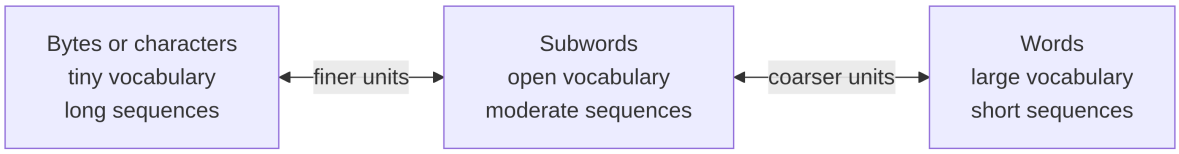


Figure 7: Token granularity trades sequence length against vocabulary size and lexical coverage.

Tokenizers can also distribute capacity unevenly across languages. Controlled experiments found that replacing a multilingual model's tokenizer with a language-specialized tokenizer improved downstream performance for almost every studied language/task combination (Rust et al. 2021). We will return to BPE, WordPiece, and model vocabularies when we study pretrained language models. For now, retain the key idea: **a tokenizer is part of the model**.

6 Regular expressions as bounded recognizers

A regular expression (regex) describes a set of string patterns. Regex is excellent for local, explicit structures such as course codes, whitespace runs, simple identifiers, or candidate URLs. It is not a general parser for human language.

The Python `re` module embeds this small pattern language inside Python (Python Software Foundation 2026). Four ideas cover much practical use:

Table 3: Core regular-expression constructs. Raw Python strings keep backslashes readable.

Construct	Meaning	Example
Character class	One character from a set	`[A-Z]`, `\d`
Quantifier	Repetition	`+`, `*`, `{2,4}`
Group	Treat subpatterns as a unit	`(NLP|CL)`
Boundary/anchor	Position rather than a character	`\b`, `^`, `$`

Suppose course identifiers follow `CS 351`, with optional extra spaces and case-insensitive letters:

```
import re

course_code = re.compile(r"\bCS\s+\d{3}\b", re.IGNORECASE)
assert course_code.findall("cs 351 and CS 410") == ["cs 351", "CS 410"]
```

Read the pattern left to right:

```
\b       CS       \s+         \d{3}       \b
edge     letters  whitespace  3 digits     edge
```

6.1 Why \w+ is not a universal tokenizer

In Python string patterns, `\w` is Unicode-aware and includes Unicode alphanumeric characters plus underscore, unless ASCII-only behavior is requested (Python Software Foundation 2026). That is useful, but “Unicode alphanumeric” is not equivalent to “word in every language.” It does not encode task-specific policies for apostrophes, emoji sequences, URLs, or clitics.

A small scanner can combine protected patterns with a fallback rule:

```
import re

TOKEN_PATTERN = re.compile(
    r"""
    https?://[^\s]+                  # URL candidate
    |[\w.+-]+@[\w.-]+\.[A-Za-z]{2,} # email candidate
    |\w+(?:['’]\w+)*                 # word with apostrophes
    |[^\w\s]                         # one punctuation/symbol
    """,
    re.VERBOSE | re.UNICODE,
)

def regex_tokenize(text: str) -> list[Token]:
    return [Token(m.group(), m.start(), m.end()) for m in
     ↪  TOKEN_PATTERN.finditer(text)]
```

This is a **teaching tokenizer**, not a universal one. Its URL rule may absorb trailing punctuation; its email grammar is incomplete; and its treatment of emoji is naïve. Those limitations should become tests, not footnotes we forget.

> ⚠ **Regex has a scope**
>
> Use regex when the pattern is local and the accepted language can be described clearly. Use a tokenizer or parser when decisions require extensive linguistic context, nested structure, or maintained language-specific rules.

7 From rules to an industrial tokenizer

spaCy illustrates a maintainable rule-based tokenization architecture. Its tokenizer first handles whitespace-separated spans, checks special cases, and iteratively processes prefixes, suffixes, and infixes while protecting patterns such as URLs. Exceptions can encode forms whose tokenization differs from the general rules (Explosion 2026).

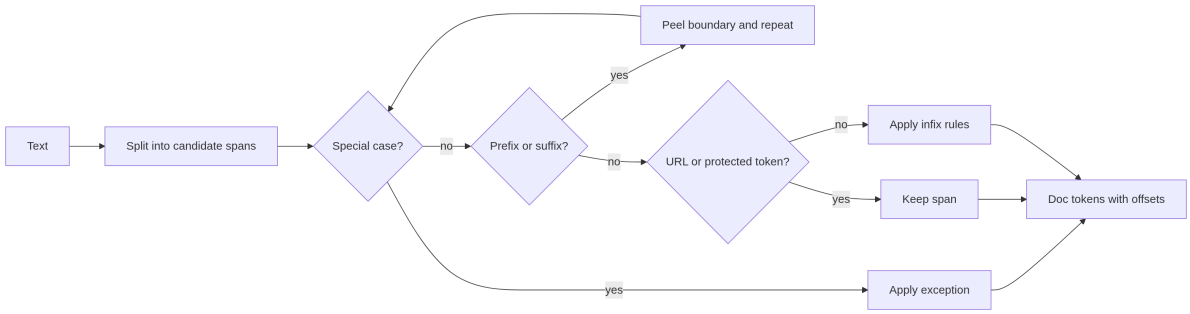


Figure 8: A simplified view of spaCy's non-destructive tokenization strategy.

```
import spacy

nlp = spacy.blank("en")
doc = nlp("I can't visit example.org.")

for token in doc:
    print(token.text, token.idx)
```

The important design is not the particular output. It is that the original text is preserved: spaCy stores whether whitespace followed each token, so concatenating token text and recorded whitespace reconstructs the input. This non-destructive property is valuable whenever annotations must map back to what a user actually wrote.

7.1 Compare tokenizers with adversarial examples

A useful evaluation set is small, diverse, and difficult on purpose:

- `I can't attend.`
- `Email nlp@example.org.`

- `Version 3.10.18 is installed.`
- Arabic: رائع جدًا!
- Chinese: 中文分词并不依赖空格。
- A woman-technologist emoji followed by `builds NLP systems.`

For each example, inspect:

- token strings;
- start and end offsets;
- reconstruction of the original;
- behavior of protected spans;
- stability under the chosen normalization policy; and
- suitability for the downstream task.

8 A reproducible preprocessing pipeline

The goal of a preprocessing pipeline is not merely to return a list of strings. It should make transformations explicit, preserve evidence, and support tests.

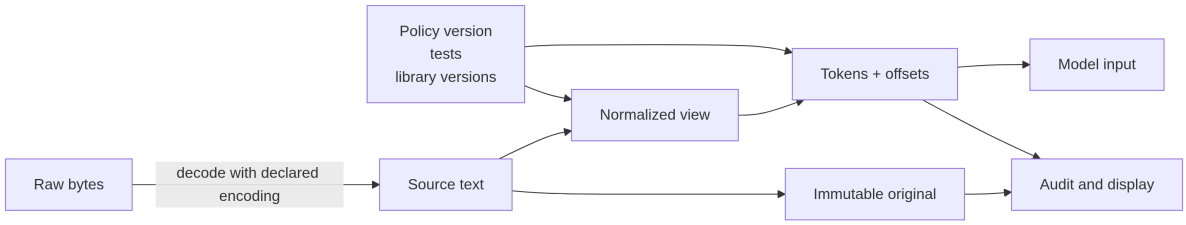


Figure 9: A robust pipeline keeps immutable source text beside derived representations and their provenance.

8.1 Invariants worth testing

An **invariant** is a property that should remain true across all accepted inputs. Strong preprocessing code makes these properties executable:

```
def test_tokens_are_ordered_and_non_overlapping():
    text = "NLP is fun!"
    tokens = regex_tokenize(text)
    assert all(a.end ⩽ b.start for a, b in zip(tokens, tokens[1:]))

def test_offsets_recover_surface_tokens():
    text = "NLP is fun!"
    for token in regex_tokenize(text):
        assert text[token.start:token.end] == token.text

def test_nfc_is_idempotent():
    text = "cafe\N{COMBINING ACUTE ACCENT}"
    once = unicodedata.normalize("NFC", text)
    twice = unicodedata.normalize("NFC", once)
    assert once == twice
```

Useful invariants include:

- decoding errors are explicit rather than silently replaced;
- normalization is idempotent: $N(N(x)) = N(x)$;
- tokens are ordered and non-overlapping;
- each token offset retrieves the recorded surface span;
- protected spans remain intact;
- empty or whitespace-only input has documented behavior; and
- the original input is retained.

8.2 Keep policy separate from mechanism

The function implementing normalization is a mechanism. Whether to case-fold is policy. Separating them makes experiments comparable and prevents an undocumented change from silently altering the dataset.

A minimal experiment record might include:

```yaml
preprocessing:
  encoding: utf-8
  unicode_normalization: NFC
  casefold: false
  tokenizer: regex-tokenizer-v1
  preserve_offsets: true
```

Version the policy with the code and tests. A project manager such as `uv` can lock Python dependencies, but a lockfile alone does not record preprocessing choices.

9 Worked case study: a noisy multilingual review

Consider the source text:

> `AMAZING café!!!  ５０  سعرالDH  —  email: Info@Example.ma` *[woman-technologist emoji]*

Assume the task is sentiment classification, while reviewers must be able to inspect the original span. We can now reason rather than “clean.”

9.1 Step 1: preserve and inspect

Store the exact source. Inspect unexpected code points. The full-width digits `５０` are compatibility variants, and the emoji contains multiple code points. Neither fact is visible from a whitespace split.

9.2 Step 2: choose a normalization view

- NFC safely regularizes canonical composition.
- NFKC would turn `５０` into `50`, which may help a price feature.
- Case-folding `AMAZING` would remove emphasis that could be useful for sentiment.
- Deleting Arabic text because a Latin-oriented rule does not recognize it would be data loss, not cleaning.

A defensible design preserves the NFC source view and derives a separate NFKC search/feature view when needed.

9.3 Step 3: tokenize with protected spans

The email should remain one candidate. Repeated exclamation marks may be stored separately or as a run, but the policy should preserve the count if emphasis matters. The emoji should be segmented using grapheme-aware tooling if the application treats it as one visible unit.

9.4 Step 4: retain evidence

For every derived token, keep its surface form and source alignment. If a compatibility-normalized feature is `50`, record that it derives from the surface span `５０` rather than replacing the original text.

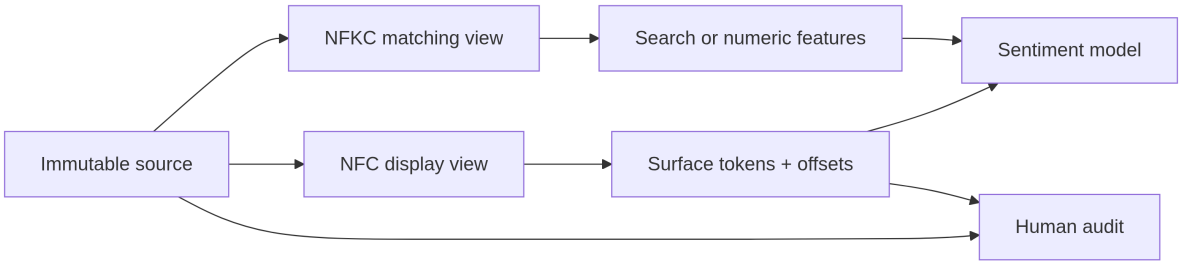


Figure 10: One source can support multiple task-specific views without destroying evidence.

This architecture costs more than calling `.lower()` and deleting punctuation. It also makes errors explainable and future changes possible.

10 Summary

1. NLP begins with representation decisions, not with a model.
2. Linguistic ambiguity means visible form does not uniquely determine meaning.
3. Bytes, code points, grapheme clusters, and tokens are different units.
4. Unicode normalization resolves specified equivalences; it is not generic text cleaning.
5. Compatibility normalization can be useful and lossy at the same time.
6. Tokenization is a task- and language-dependent contract.
7. Regular expressions are effective bounded recognizers, not universal language parsers.

8. Token offsets, immutable source text, explicit policies, and invariants make preprocessing reproducible.

Practical checklist

Before accepting a preprocessing pipeline, ask:

- What encoding is expected, and how are decoding errors handled?
- Which normalization form is applied, and why?
- Which distinctions are intentionally removed?
- What is a token for this task and domain?
- Are multilingual and adversarial examples included in tests?
- Can every processed span be traced back to the original input?
- Are the policy and dependency versions recorded?

11 Exercises

11.1 Conceptual questions

1. Explain why the number of Python code points in a string may differ from the number of symbols a user perceives.
2. Give a task where lowercasing is helpful and another where it may remove signal.
3. Why can two valid tokenizers disagree without either being universally wrong?
4. Explain the difference between canonical and compatibility equivalence.
5. Why should a model prediction be stored with source offsets?

11.2 Unicode investigation

Write a function that prints the UTF-8 bytes, code points, Unicode names, and NFC/NFD forms of a string. Test it on:

- precomposed and decomposed é;
- Arabic text with and without diacritics;
- , fi, and full-width digits;
- an emoji containing a skin-tone modifier or ZERO WIDTH JOINER.

For each example, state which transformations preserve the information needed by a task you choose.

11.3 Tokenizer experiment

Implement the teaching regex tokenizer from this chapter and compare it with spaCy on at least twenty adversarial examples. Organize failures into categories such as contractions, URLs, numeric expressions, abbreviations, Arabic clitics, CJK text, and emoji. Propose two rule changes and add regression tests before implementing them.

11.4 Pipeline design

Design a preprocessing policy for one of these applications:

- multilingual sentiment analysis;
- exact legal-document search;
- social-media named-entity recognition; or
- duplicate product-title detection.

Document what you preserve, normalize, tokenize, and discard. For every destructive operation, explain why the lost distinction is irrelevant or acceptably risky for the chosen task.

References

Explosion. 2026. *Linguistic Features: Tokenization*. https://spacy.io/usage/linguistic-features#tokenization.

Jurafsky, Daniel, and James H. Martin. 2026. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*. 3rd ed. https://web.stanford.edu/~jurafsky/slp3/.

Mielke, Sabrina J., Zaid Alyafeai, Elizabeth Salesky, et al. 2021. "Between Words and Characters: A Brief History of Open-Vocabulary Modeling and Tokenization in NLP." *Transactions of the Association for Computational Linguistics* 9: 1572–85. https://doi.org/10.1162/tacl_a_00429.

Python Software Foundation. 2026. *Regular Expression HOWTO*. https://docs.python.org/3/howto/regex.html.

Rust, Phillip, Jonas Pfeiffer, Ivan Vulić, Sebastian Ruder, and Iryna Gurevych. 2021. "How Good Is Your Tokenizer? On the Monolingual Performance of Multilingual Language Models." *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing*, 3118–35. https://doi.org/10.18653/v1/2021.acl-long.243.

Sennrich, Rico, Barry Haddow, and Alexandra Birch. 2016. "Neural Machine Translation of Rare Words with Subword Units." *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, 1715–25. https://doi.org/10.18653/v1/P16-1162.

Unicode Consortium. 2025a. *Unicode Normalization Forms*. Unicode Standard Annex No. 15. Unicode Consortium. https://unicode.org/reports/tr15/.

Unicode Consortium. 2025b. *Unicode Text Segmentation*. Unicode Standard Annex No. 29. Unicode Consortium. https://unicode.org/reports/tr29/.